

## **Abstract**

This report focuses on RISC-V architecture and its emulation using QEMU. It covers the core principles of RISC, the structure of RISC-V registers and instruction formats, and how these contribute to efficient processing. It explains how QEMU uses dynamic binary translation and Just-In-Time compilation to emulate RISC-V CPUs without physical hardware. The report also touches on Edge-AI for local, intelligent processing and reduced network dependency. Finally, it discusses fault injection as a method to test system behavior under error conditions.

## **Introduction:**

RISC, or *Reduced Instruction Set Computer*, is a CPU design philosophy that focuses on a smaller, highly optimized set of instructions (Patterson and Hennessy, 2017). This lean instruction set makes RISC processors faster and more efficient compared to CISC (Complex Instruction Set Computer) processors, which rely on a larger and more complex set of instructions. In RISC-based CPUs, most instructions can be completed in just one clock cycle, even if achieving a certain task requires more instructions overall. This simplicity also makes RISC architectures easier to design and implement.

This report outlines the basics of RISC-V architecture and how QEMU emulates RISC-V environments for testing without requiring physical hardware. It covers edge AI for executing lightweight AI tasks locally and explores fault injection techniques to evaluate the resilience and efficiency of edge AI systems under real-world faults. These sections provide a foundation for simulating fault-tolerant AI workloads on RISC-V using QEMU.

## **Principles of RISC Architecture:**

All RISC CPU designs follow five core principles.

1. **Simple Instructions:** Each instruction is designed to be executed in a single clock cycle.
2. **Fixed Instruction Format:** All instructions follow a consistent, fixed structure.
3. **Load-Store Model:** Only specific instructions handle memory access; all other operations are performed between registers.
4. **Large Register Set:** A bigger pool of general-purpose registers helps minimize memory access.
5. **Pipelining:** Simple and uniform instructions allow multiple instructions to be processed simultaneously using pipelining.

## **RISC-V:**

RISC-V is a free, open-source RISC architecture introduced in 2010 by researchers at the University of California, Berkeley (Waterman and Asanovic, 2017). Since then, it has gained popularity across academic, industrial, and embedded system communities. RISC-V sticks to RISC design principles while offering modularity, making it suitable for everything from low-power embedded systems to high-performance computing.

## **CPU Registers in RISC-V:**

Registers are small, fast storage units inside the CPU used to store operands and intermediate results during computation. Since accessing registers is significantly faster than accessing memory, RISC-V encourages heavy register use for performance reasons.

## **Number of Registers:**

RISC-V offers three main base architectures:

- RV32I includes 32 registers, each 32 bits wide.
- RV64I has 32 registers, but each is 64 bits wide.
- RV128I features 32 registers, each 128 bits wide.

Of these 32 registers, 31 serve as general-purpose registers, and one, known as the zero register, always holds the value 0 (logical false).

If the processor supports floating-point and double-precision operations, RISC-V adds a dedicated set of floating-point registers to handle tasks such as floating-point addition, multiplication, and division.

## **RISC-V Instruction Set:**

The core goal of RISC-V is to offer a streamlined set of instructions that can mostly be executed in a single cycle. These instructions are divided into categories:

**Arithmetic Instructions:** Basic math using registers (e.g., ADD, SUB, MUL, DIV).

**Logical Instructions:** Bit-level operations like AND, OR, XOR, NOT.

**Memory Access Instructions:** Moving data between registers and memory, like LW (load word) and SW (store word).

**Control Flow Instructions:** Managing the flow of program execution, including BEQ (branch if equal) and JAL (jump and link).

## **Instruction Formats**

RISC-V uses several instruction formats, mainly R-type, I-type, and S-type. Each format defines how to place the opcode, register references, and immediate values within a 32-bit instruction.

## **Fixed-Length Simplicity**

All standard RISC-V instructions are 32 bits long. This fixed size streamlines instruction decoding and simplifies CPU pipeline design, improving execution speed and efficiency.

## **Opcode and Immediate Values**

Each RISC-V instruction includes an opcode, which tells the CPU what operation to perform. Some instructions also use immediate values, which are constant values embedded directly in the instruction.

## **Special Instructions**

RISC-V includes atomic and privileged instructions for tasks like synchronization and system control. For instance, FENCE enforces memory access to maintain proper execution behavior.

## **QEMU:**

Quick EMulation, QEMU for short, is an open-source machine emulator and virtualizer that lets the user emulate various CPU architectures, CPU peripherals, software, and operating systems on a host machine that may or may not be suitable to run the target machine directly. QEMU allows users to test and develop architectures, peripherals, firmware, and software without requiring physical hardware.

## **Emulation and Virtualization:**

QEMU can emulate the behavior of various architectures, peripherals, and software, including operating systems (Bellard, 2005). QEMU operates by using the machine code designed for the target machine while running on a host machine having a different machine code. Additionally, QEMU can create multiple emulation environments on a single host machine.

## **CPU emulation on QEMU:**

QEMU emulates RISC-V through the "Dynamic Binary Translation (DBT)" method. QEMU takes in the machine code of the target CPU architecture. Through DBT, QEMU translates the target's machine code into the machine code the host machine can understand and execute. QEMU uses Just-In-Time (JIT) compilers to translate as the host machine continues executing the translated codes. Using JIT ensures faster and more efficient translation and execution.

## **RISC-V Emulation in QEMU:**

QEMU emulates RISC-V in the following steps (Bellard, 2005):

1. **Instruction Set Simulation:** QEMU interprets the target RISC-V architecture in the host system's instructions set. This interpretation supports the full range of instructions set for normal operations and optional ones for Floating-Point and Double-Point units.
2. **Memory Management:** QEMU emulates the memory model of the RISC-V architecture, memory access, and memory mapping to ensure smooth and continuous operation throughout the emulation period.
3. **QEMU can emulate RISC-V peripherals,** timers, serial interfaces, and interrupts. Peripheral emulation allows for a more comprehensive simulation of the target architecture without requiring a physical setup.

## **Performance Optimization:**

QEMU uses Just-In-Time (JIT) compilers to translate the target's machine code into the host's native language. JIT compilers save the resources and time that would waste in making the host machine interpret or execute the target's machine code directly. As a result, the emulation environment can better utilize the host's resources.

## **Edge-AI**

Edge-AI refers to using algorithms and artificial intelligence to enhance the autonomy of digital and IoT devices (Li et al., 2020). The primary goals of Edge-AI are:

1. **Enhanced Autonomy:** Edge-AI aims at making digital and IoT devices less reliant on network connectivity.
2. **Local Processing:** Edge-AI aims at using local or available resources to operate and make decisions using artificially intelligent algorithms for maximum efficiency and autonomy.

3. **Maximizing efficiency** through local resources and AI algorithms reduces latency, increases bandwidth, and accelerates decision-making.
4. **Improved Security:** Edge-AI aims at making IoTs and digital devices more secure against cyberattacks and potential data breaches by making these devices less dependent on network connectivity.

### **A complete QEMU command looks like this:**

*qemu-system-riscv64 -machine virt -cpu rv64 -kernel /path/to/your/kernel/image> -nographic*

### **Breakdown:**

1. The **-machine** option allows users to define the hardware configuration, peripherals, other devices, or the board they are trying to emulate. For example, `-machine virt` defines a generic virtual machine with default peripherals.
2. The **-cpu** option lets users specify a CPU model to emulate. Users can select from a wide variety of architectures and specify the CPU's features. Example: `-cpu rv64` tells QEMU that we are emulating a 64-bit RISC-V CPU.
3. The **-kernel** option is used to load the required kernel image for the emulation. For RISC-V emulation, the typical kernel image is a Linux image. Example: `-kernel <path to the kernel image>`.
4. The **-nographic** option disables any graphical output and redirects all outputs to the terminal. As a result, users see the outputs in the terminal instead of in a new window.

### **Fault Injection:**

Fault Injection refers to the deliberate introduction of faults into a system to observe how it reacts and what warnings are generated in response to those faults (Koopman, 2016). It can reveal how the system identifies and rectifies errors.

### **Types of Fault Injection:**

There are three main types of fault injection:

**Transient Faults:** Transient faults, also known as soft errors, occur briefly and are typically caused by changes in the physical environment. For example, cosmic rays or fluctuations in the electromagnetic field can result in bit-flips in memory or registers, leading to abnormal system behavior. However, transient faults usually disappear when environmental conditions return to normal.

**Permanent Faults:** Permanent faults arise when the system has physically defective hardware or critical errors in its instruction set or programming. Unlike transient faults, permanent faults require human intervention for resolution. For instance, a bent CPU pin can lead to a defective CPU that consistently shows a value stuck at '0' or '1'.

**Intermittent Faults:** Intermittent faults are more challenging to replicate in emulation or simulation environments. These faults can occur when voltage levels across a component are slightly above or below the rated voltage, also known as borderline voltage. Such faults can be difficult to reproduce and often vanish after a system reboot.

### **Fault Injection Techniques**

**Hardware Fault Injection:** Hardware fault injection introduces faults directly in the system's hardware components. For example, cutting power to a hardware component, disconnecting it while the system is running, or introducing a voltage spike to the CPU can help determine if the system can detect and correct faults effectively.

**Software Fault Injection:** Software fault injection deliberately introduces faults in the software or the instructions set for the CPU to manipulate system states, memory, or inputs. For example, introducing a segmentation fault in the code to force a fault in the system and see how the system handles the fault and if it can initiate a secured failure.

**Virtual Fault Injection:** The Virtual fault injection method uses emulation software like QEMU to emulate and inject faults into a virtual machine without requiring physical components.

## **Why is Fault Injection Important:**

### **Testing System Resilience:**

Fault injection helps expose how a system reacts under stress. By deliberately introducing faults, it observes whether the system crashes, recovers, or behaves unexpectedly. These reactions help identify weak points and improve overall robustness. Instead of relying on assumptions, it tests actual behavior.

### **Improving Fault Tolerance:**

For critical systems—like in aerospace, medical devices, or autonomous machines—fault tolerance is non-negotiable. It injects both soft and hard faults to see if the system absorbs the hit or breaks. Each failure reveals a path to improvement. It uses this process to harden systems before real-world deployment.

### **Enhancing Security:**

Fault injection can also uncover hidden security vulnerabilities. By disrupting execution, memory, or control flow, it checks if faults let unintended behavior slip through. This method helps detect potential attack surfaces, especially in secure environments, and ensures the system doesn't break under tampering.

### **Verifying Safety-Critical Applications:**

Systems used in safety-critical scenarios must behave predictably, even during faults. It uses fault injection to validate whether the system fails safely, sends alerts, or maintains core functionality. These insights support the development of reliable systems where safety cannot be compromised.

### **Energy Optimization for Edge AI:**

Fault injection doesn't just test system reliability—it also uncovers how to make the process more efficient. Most edge-AI devices operate under energy constraints. Fault injection helps reveal inefficiencies like unnecessary computations, retries, or instability in the model. Techniques like early fault detection, adaptive model behavior, and redundancy management help reduce energy waste and make better use of resources. Evaluating energy metrics allows developers to fine-tune algorithms and AI behavior for the most efficient and long-lasting performance (De Matteis et al., 2020).

# **Setting Up RISC-V Environment for QEMU on WSL2**

## **Step 1: Enable WSL 2 and Install Ubuntu**

Open PowerShell as Administrator and run:

```
dism.exe /online /enable-feature /featurename:Microsoft-Windows-Subsystem-Linux /all /norestart
```

```
dism.exe /online /enable-feature /featurename:VirtualMachinePlatform /all /norestart
```

Restart your system.

Install Ubuntu from Microsoft Store (Ubuntu 24.04 LTS).

## **Step 2: Update System Packages**

Open Ubuntu terminal and run:

```
sudo apt update && sudo apt upgrade -y
```

## **Step 3: Install Required Packages**

Install the necessary tools:

```
sudo apt install build-essential git curl gcc g++ make libncurses-dev bison flex libssl-dev libelf- dev
```

```
qemu-system-misc qemu-system-riscv -y
```

## **Step 4: Download and Extract Kernel Source**

Download and extract Linux kernel:

```
mkdir -p ~/kernel-build && cd ~/kernel-build
```

```
curl -LO https://cdn.kernel.org/pub/linux/kernel/v6.x/linux-6.14.5.tar.xz
```

```
tar -xvf linux-6.14.5.tar.xz && cd linux-6.14.5
```

## **Step 5: Build the Linux Kernel for RISC-V**

Configure and build the kernel:

```
make ARCH=riscv defconfig
```

```
make -j$(nproc) ARCH=riscv CROSS_COMPILE=riscv64-linux-gnu-
```

## **Step 6: Download Ubuntu RootFS Image for RISC-V**

Download prebuilt image

```
https://cdimage.ubuntu.com/releases/24.04/release/ubuntu-24.04.2-preinstalled-server-riscv64+nez
```

```
ha.img.xz
```

```
unxz ubuntu-24.04.2-preinstalled-server-riscv64+nezha.img.xz
```

## **Step 7: Start QEMU with VNC Enabled**

Use the following command:

```
qemu-system-riscv64 -machine virt -cpu rv64 -m 2G \
```

```
-kernel ~/kernel-build/linux-6.14.5/arch/riscv/boot/Image.gz \
```

```
-drive file=~/ubuntu-24.04.2-preinstalled-server-riscv64+nezha.img,format=raw,if=virtio \
```

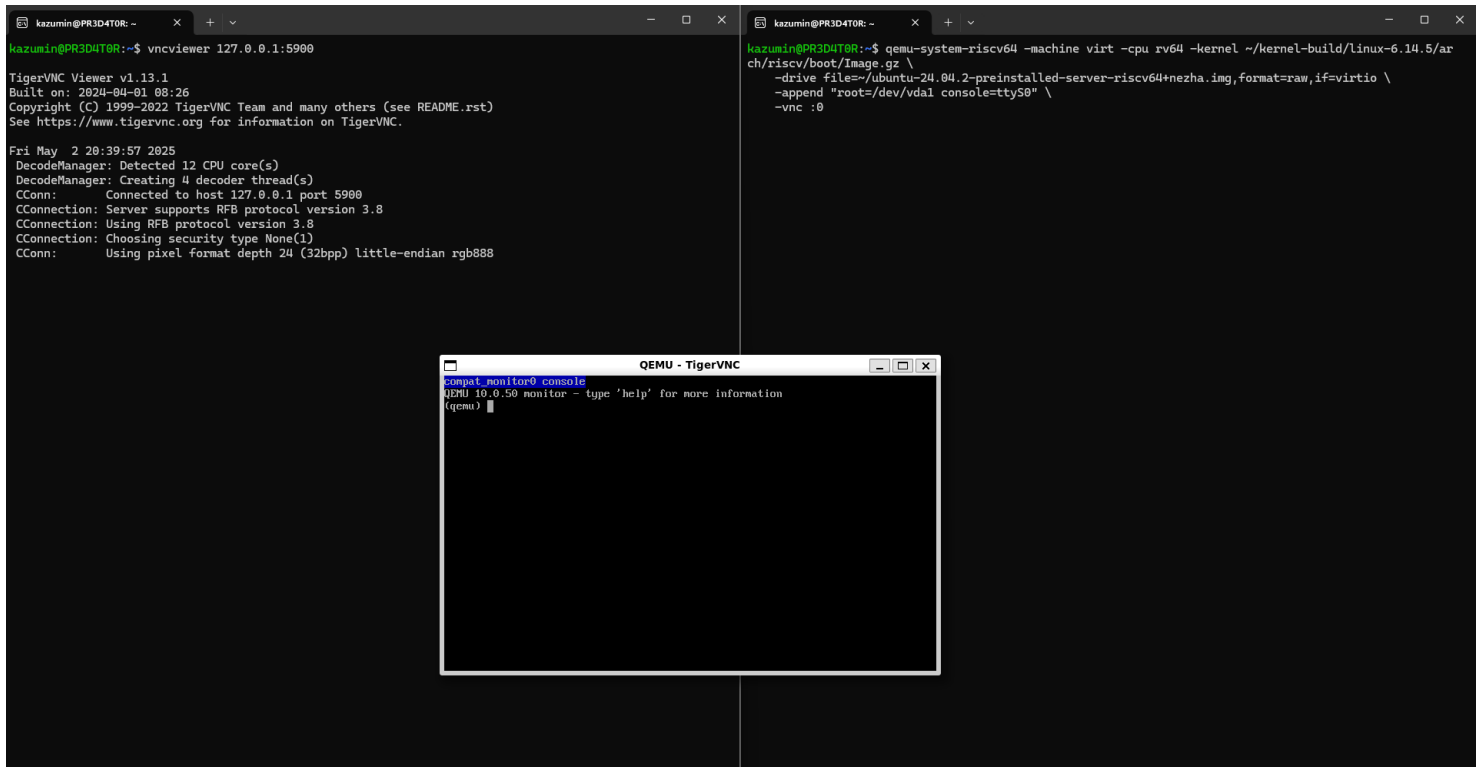
```
-append "root=/dev/vda1 console=ttyS0" \
```

```
-vnc :0
```

## **Step 8: Connect to QEMU with TigerVNC**

Open TigerVNC viewer and connect to:

```
localhost:5900
```



## Conclusion

RISC-V simplifies CPU architecture by sticking to the essentials—fixed instruction lengths, a load-store model, and a large register set—making it ideal for efficient computing. Through QEMU, we can emulate and test RISC-V environments without needing the actual hardware. This opens the door for safe, cost-effective development and testing. Edge-AI pushes this further by reducing dependency on the cloud and boosting autonomy at the device level. Fault injection plays a critical role in identifying how systems respond to errors, helping to make designs more reliable and secure. Together, these tools and principles form a strong foundation for modern embedded systems development.

## **Bibliography**

- [1] D. A. Patterson and J. L. Hennessy, Computer Organization and Design RISC-V Edition: The Hardware Software Interface, 1st ed. San Francisco, CA, USA: Morgan Kaufmann, 2017.
- [2] A. Waterman and K. Asanovic, The RISC-V Instruction Set Manual, Volume I: User-Level ISA, Version 2.2, EECS Department, University of California, Berkeley, 2017.
- [3] F. Bellard, QEMU, a Fast and Portable Dynamic Translator, in USENIX Annual Technical Conference, 2005.
- [4] D. Li, Z. Zhou, and X. Chen, Edge AI: On-demand accelerating deep neural network inference via edge computing, IEEE Transactions on Wireless Communications, vol. 19, no. 1, pp. 447457, Jan. 2020.
- [5] P. Koopman, Understanding fault injection, IEEE Security & Privacy, vol. 14, no. 4, pp. 7175, Jul.-Aug. 2016.
- [6] T. De Matteis, M. R. Kakoei, A. Capotondi, A. Pullini, and L. Benini, “Energy-efficient neural network inference on RISC-V cores with fault tolerance,” IEEE Transactions on Circuits and Systems I: Regular Papers, vol. 67, no. 11, pp. 3881–3894, Nov. 2020.